Project 3 – Differential-drive robot

Martina De Angelis, David Grander

AI was used as a coding assistant

1. Introduction

This report describes the design and implementation of a navigation stack for a differential-drive mobile robot operating in a planar, obstacle-cluttered arena. The robot must move from a start pose to a goal pose while avoiding collisions. It is equipped with a GPS sensor that measures only its planar position, so the orientation must be deduced. The three tasks addressed here are: (1) a motion planner producing a collision-free reference trajectory, built from an optimization-based local planner, a collision-checking routine, and a probabilistic roadmap (PRM) global planner; (2) a feedback controller that tracks the reference trajectory; and (3) a state estimator that fuses the noisy GPS measurements with the known control inputs to localize the robot. Throughout, the robot footprint is abstracted as a disc and the obstacles as capsules, which keeps collision queries cheap and analytic.

The system is implemented in Python on top of the provided Pinocchio/Meshcat simulation environment. CasADi with the IPOPT solver is used for the optimization-based planner. The remainder of the report follows the natural data flow of the stack: the model and environment, then planning, then control, and finally estimation, closing with validation and the transition toward the car-like robot of Task 4.

2. Robot model and simulation environment

The differential-drive robot is described by the unicycle kinematic model, with the configuration $q = (x, y, \theta)$ and the control input $u = (v, \omega)$, where v is the forward linear velocity and ω the angular velocity. The continuous-time kinematics are

$$\dot{x} = v\cos\theta, \quad \dot{y} = v\sin\theta, \quad \dot{\theta} = \omega$$

These are integrated with a forward-Euler scheme at a fixed time step Δt to obtain the discrete dynamics used everywhere in the project:

$$x_{k+1} = x_k + v_k cos\theta_k \Delta t$$

$$y_{k+1} = y_k + v_k sin\theta_k \Delta t$$

$$\theta_{k+1} = \theta_k + \omega_k \Delta t$$

The heading is wrapped to (−π, π] after every update using the $atan2(\sin\theta, \cos\theta)$ trick, which avoids discontinuities and keeps angular errors well defined. The simulation step

uses a small $\Delta t = 0.01s$ for a smooth, physically faithful rollout, whereas the planner discretizes its horizon more coarsely (see Section 3.1) for tractability.

To make the simulation realistic and to give the estimator something to do, an actuation disturbance is injected at every step: independent zero-mean Gaussian noise (standard deviation 0.1) is added to both v and ω before integration. This represents wheel slip and command tracking error and is the dominant source of dead-reckoning drift. The GPS returns the true position corrupted by zero-mean Gaussian noise with standard deviation 0.03m on each axis, and never observes orientation. A deliberate design decision was to make the disturbance standard deviations explicit constants so that the estimator's process-noise model can be matched to them exactly (Section 5).

The arena is a 10×10m square bounded by four walls, with a cylindrical tower in each corner. Internal clutter consists of cylindrical pillars; pairs of pillars are joined by rectangular walls. This pillar-plus-connecting-wall construction is exactly a capsule: a line segment inflated by a radius. Representing obstacles as capsules is what makes the collision check (Section 3.2) reduce to point-to-segment and point-to-point distances. The robot's rectangular 0.5×0.3m footprint is enclosed by a bounding disc of radius about 0.31m; using a single conservative disc radius means the robot can be treated as a point against inflated obstacles, which is both fast and orientation-independent.

3. Task 1: Motion Planning

The planner is organized as three cooperating components: a local planner that steers between two poses ignoring obstacles, a collision check that validates any candidate path, and a PRM global planner that stitches local plans into a collision-free route across the whole arena. This is the structure suggested in Section 10.5.2 of Modern Robotics, with the local planner playing the role of the roadmap's steering subroutine.

3.1 Local Planner (optimization-based)

The local planner is formulated as a nonlinear program solved with CasADi and IPOPT. The decision variables are the full state trajectory X of shape $3 \times (N+1)$ and the control trajectory U of shape $2 \times N$, with horizon $N = 20$ and planning step $\Delta t = 0.2\ s$. Rather than rolling the dynamics out from the controls, we use a multiple-shooting transcription: the states are free variables and the kinematics enter as equality constraints linking consecutive knots,

$$X_{k+1} = f(X_k, U_k), \;\; k = 0, \ldots, N-1$$

This keeps the constraint Jacobians sparse and well conditioned and lets us impose the start and goal poses directly as boundary constraints $X_0 = q_{start}$ and $X_N = q_{goal}$. The

control inputs are bounded by $v \in [0, 1.5] m/s$ and $\omega \in [-1.5, 1.5] rad/s$. Restricting the linear velocity to be non-negative was a conscious choice: it forbids reversing, so the local planner always produces forward-driving, intuitive manoeuvres rather than exploiting the symmetry of the unicycle to back into the goal.

To ensure smooth and practical motion, the cost function minimizes three main components over the horizon:

- linear effort v^2 with a small weight (1.0), which mildly discourages unnecessarily fast motion;
- angular effort ω^2 with a large weight (20.0), which strongly discourages spinning and biases the solution toward straight, low-curvature paths;
- the angular-rate change $(\omega_{k+1} - \omega_k)^2$ with weight 10.0, which smooths the steering profile and prevents abrupt heading switches.

The heavy relative weighting of angular terms is the key design lever: because turning is expensive and jerky steering is penalized, the planner returns smooth arcs that a non-holonomic robot can follow comfortably, instead of mathematically valid but practically poor zig-zag solutions. The solver is warm-started with a sensible initial guess: x and y are linearly interpolated between start and goal, the heading is interpolated along the shortest angular arc (computed via $atan2(sin\, \Delta\theta, cos\, \Delta\theta)$ to avoid the 2π wrap-around), and the controls are seeded with a small constant. If IPOPT fails to converge the planner returns a failure flag, which the global planner treats as an absent edge.

3.2 Collision Check

The collision check takes a discretized path and returns true only if every sampled configuration is clear of all obstacles by a safety margin. Thanks to the capsule abstraction, each query is analytic. For the cylindrical pillars and corner towers, modelled as circles, the test is a point-to-centre distance compared against the sum of the robot radius, the obstacle radius and the safety margin. For the outer walls and the connecting walls between pillars, modelled as line segments, the test uses a point-to-segment distance: the path point is projected onto the segment, the projection parameter is clamped to [0, 1] so that the nearest point stays on the segment, and the resulting distance is compared against the robot radius (plus the wall half-width for the connecting walls) and the margin. A safety margin of 0.08m is added to every threshold so that paths keep a buffer from obstacles and remain robust to the tracking error introduced later by control and estimation. Because the check is applied to the dense set of knots produced by the local planner, sampling between knots is unnecessary in practice.

3.3 Global Planner (PRM with A)*

The global planner is a probabilistic roadmap. Configurations are sampled uniformly at random inside the arena and retained only if collision-free; the start and goal poses are always inserted as nodes. Each node is then connected to its nearest neighbours (up to a fixed count, and only within a maximum radius) by invoking the local planner as the steering subroutine. An edge is admitted only when the local plan exists, passes the collision check, and is not degenerate.

Degeneracy is detected with a path-length test: if the arc length of a candidate edge exceeds three times the straight-line distance between its endpoints, the edge is discarded as a looping or spinning solution. This filter prevents the roadmap from being polluted by local-planner solutions that technically connect two nodes but do so with an unnatural detour. For efficiency, every accepted local path is cached together with its reversed version, so the same edge is never re-solved in the opposite direction.

The shortest route through the roadmap is found with A* rather than plain Dijkstra. The edge weights are Euclidean distances between sampled positions, and the heuristic is the straight-line distance from a node to the goal. Since that heuristic never overestimates the true remaining cost it is admissible, so A* returns the same optimal path as Dijkstra while expanding fewer nodes.

A raw roadmap path can still have abrupt orientation changes at the waypoints, because the intermediate node headings were sampled randomly. The path is therefore post-processed: at each interior waypoint the heading is reset to point toward the next waypoint, and every segment is re-solved with the local planner using these aligned headings (again normalizing the angular difference to the shortest arc). If a re-solved segment loops or collides, the routine falls back to the cached original edge, guaranteeing that a valid segment is always available. Finally a continuity check rejects any path containing a hairpin turn (a heading jump larger than 90° between consecutive segments) and asks for a re-run, because such a discontinuity cannot be tracked smoothly by the controller. The accepted segments are concatenated into the reference state trajectory X_{ref} and reference control trajectory U_{ref} that are handed to the controller.

4. Task 2: Trajectory-tracking controller

The controller follows the nonlinear feedback design for a unicycle given in Section 13.3.4 of Modern Robotics, combining feedforward of the reference controls with feedback on the

tracking error. The error between the reference pose and the current pose is first expressed in the robot's own body frame:

$$e_1 = cos\theta \cdot e_x + sin\theta \cdot e_y$$

$$e_2 = -sin\theta \cdot e_x + cos\theta \cdot e_y$$

$$e_\theta = wrap(\theta_{ref} - \theta)$$

where $(e_x, e_y) = (x_{ref} - x,\ y_{ref} - y)$. Here e_1 is the along-track error (ahead/behind), e_2 is the cross-track error (left/right), and e_θ is the heading error. The control law is

$$v = v_{ref} cos(e_\theta) + k_1 e_1$$

$$\omega = \omega_{ref} + k_2 e_2 + k_3 \sin(e_\theta)$$

The feedforward terms v_{ref} and ω_{ref} keep the robot moving along the trajectory at the planned speed, while the feedback terms correct deviations: k_1 drives the along-track error to zero, and k_2 together with k_3 steers the robot back onto the path and aligns its heading. The gains used are $k_1 = 2, k_2 = 1, k_3 = 2$. The commanded velocities are saturated to the actuator limits before being applied, so the controller cannot request infeasible inputs even during large transient errors.

Selecting which point of the reference to track is handled by a carrot, or pure-pursuit-style, lookahead with a monotonic progress index. Instead of searching the whole trajectory for the nearest reference point, the search is restricted to a forward window starting from the last index reached. The progress index is only ever allowed to advance, never to move backward. This is an important robustness choice: on trajectories that pass close to themselves, a global nearest-point search could snap the target back to an earlier part of the path and stall the robot, whereas the monotonic forward search guarantees steady progress toward the goal. The target itself is placed a fixed arc-length ahead of the nearest point, which yields the smooth, anticipatory tracking characteristic of pure pursuit.

Pure trajectory tracking cannot, on its own, settle a non-holonomic robot onto an exact final pose, so a dedicated endgame takes over once the robot is within 0.5m of the goal. It runs in two phases. In the first phase the robot drives toward the goal position while steering to face it, with the linear command modulated by the alignment with the goal direction so that it never stalls when poorly oriented. In the second phase, once the goal position is reached, the robot stops translating and rotates in place to the desired final heading. Decoupling position from orientation in this way lets the controller achieve both, which a single tracking law cannot guarantee for a unicycle. Arrival is declared when the position error is below

0.15m and the heading error below about six degrees. A stuck detector monitors whether the robot has stopped making progress and aborts gracefully if so.

5. Task 3: State estimation (EKF)

Because the GPS observes position but not orientation, and because the actuation noise causes dead reckoning to drift, an Extended Kalman Filter (EKF) is used to estimate the full pose (x, y, θ) by fusing the control inputs with the GPS measurements. The controller consumes the EKF estimate rather than the (in practice unavailable) true state, closing the loop through the estimator.

In the prediction step the mean is propagated with the nominal, noise-free kinematics, and the covariance is propagated using the linearization of the motion model. The state Jacobian is

$$F = \partial f/\partial q = [[1, 0, -v\ sin\theta\ \Delta t], [0, 1, v\ cos\theta\ \Delta t], [0, 0, 1]]$$

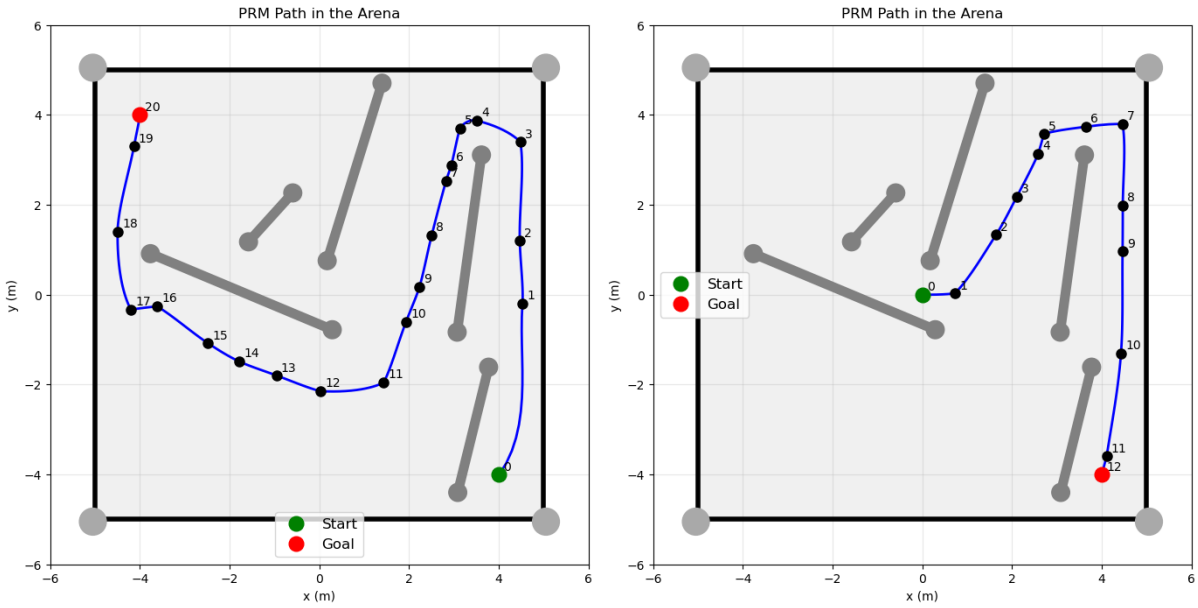
The crucial modeling choice concerns the process noise. The disturbance in the simulation enters through the control channel (it is noise on v and ω) so the filter models it the same way. With the control Jacobian

$$B = \partial f/\partial u = [[cos\theta\ \Delta t, 0], [sin\theta\ \Delta t, 0], [0, \Delta t]]$$

and the input-noise covariance $M = diag(\sigma_v^2, \sigma_\omega^2)$, the process noise is formed as $Q = BMB^{\mathrm{T}}$. The standard deviations σ_v and σ_ω are set to match the 0.1 disturbance actually injected by the simulator. Deriving Q from the control-channel disturbance, instead of bolting an arbitrary diagonal matrix onto the state, makes the filter's uncertainty grow in the directions in which the real error grows (for example, position uncertainty inflates along the current heading) which keeps the filter consistent and well tuned.

The measurement model is linear: the GPS observes position through $H = [[1, 0, 0], [0, 1, 0]]$ with measurement covariance $R = diag(\sigma_{gps}^2, \sigma_{gps}^2)$ and $\sigma_{gps} = 0.03m$. The update applies the standard Kalman equations to compute the innovation, the innovation covariance, the gain, and the corrected state and covariance, after which the heading is wrapped back to (−π, π]. The orientation, although never measured directly, becomes observable through the coupling between heading and the position increments predicted from the controls: as the robot moves, the way its measured displacement aligns (or fails to align) with the commanded motion lets the filter correct θ. The state covariance is initialized small because the robot starts from a known pose.

6. Integration and validation



The three components run together in a closed loop: at each step the GPS is sampled, the EKF update and prediction refine the pose estimate, the controller computes a command from that estimate, and the disturbed dynamics advance the true state. The full stack was validated on several start-goal pairs of varying difficulty, including routes that must thread between the connecting walls and approach the goal from a constrained direction, such as driving from the origin to a goal near a corner. In these runs the PRM reliably returns a continuous, collision-free reference, the controller tracks it with small cross-track error, and the endgame settles the robot within the 0.15m and six-degree tolerances, at which point the simulation reports success. Because the planner samples randomly, occasional runs fail the continuity check and are simply re-run, which is expected behaviour for a probabilistic roadmap.

The safety margin baked into the collision check is what makes this robustness possible: it leaves enough clearance to absorb the bounded tracking error from imperfect control and the residual estimation error, so that a path certified collision-free at planning time stays collision-free when executed under noise.

7. Task 4 : Trajectory tracking for the bicycle model

Adapting the differential-drive controller to the bicycle model exposed two coupled issues that initially prevented the robot from following the reference path. First, the PRM reference was generated by the local planner at a coarse time step (dt = 0.2 s) while the closed-loop simulation runs at dt = 0.01 s. Because the original get_reference advanced through the reference array by index, each simulation step consumed reference points spaced 0.2 s apart, so the look-ahead target jumped far ahead of the robot's achievable motion. The

tracker therefore steered toward distant carrot points rather than following the collision-checked segment geometry, which caused both the loss of path adherence and apparent passage through obstacles. We resolved this by reformulating the look-ahead as a fixed arc-length (0.5 m) measured along the path from the nearest point, making the reference selection independent of the planning time step.

Second, the controller inherited the unicycle assumption that angular velocity is directly commandable. For a car this is not the case: steering is effective only while the vehicle is moving, since the heading rate follows the bicycle relation $\omega = (v/L)\cdot\tan(\phi)$, which vanishes at $v = 0$. The original speed law allowed the commanded velocity to collapse toward zero during the initial transient, so the steering produced no turning and the vehicle stalled near the first waypoint. We addressed this by enforcing a minimum cruise speed and by deriving the feed-forward heading from the path tangent rather than from the planner's velocity outputs, which were calibrated to the wrong time step. The outer loop computes a desired angular velocity from the lateral and heading errors, which is mapped to a desired steering angle $\phi_{des} = \mathrm{atan}(L\cdot\omega_{des}/v)$; an inner servo loop then drives the steering rate command $\dot{\phi} = k_{\phi}\cdot(\phi_{des} - \phi)$, respecting the URDF steering limit of ±0.6 rad.

Finally, the "stuck" watchdog was reformulated to evaluate displacement over a 100-step window rather than per step, since per-step motion at $dt = 0.01$ s is negligible by construction and triggered false termination during normal acceleration. With these corrections the controller tracks the reference closely and reaches the goal pose. A residual limitation is that the controller itself performs no collision avoidance and relies on the reference being collision-free; where consecutive PRM segments meet at corners sharper than the steering limit permits, tracking error can grow, which is best mitigated in the planner (shorter inter-node distance or corner smoothing) rather than in the controller.